

Building data pipelines with **dbt Core** and **Snowflake**: art gallery challenge

30 September 2025

ANYA PROSVETOVA

Solutions Architect at Snowflake



Illustration by Freepik Stories

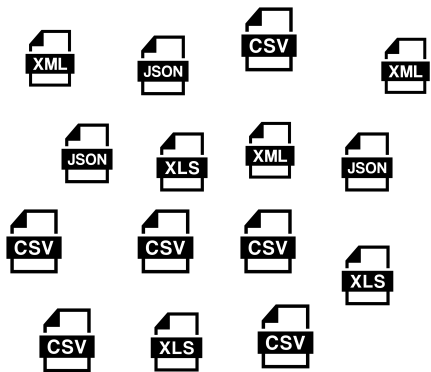
Workshop agenda

- Introduction to the art gallery challenge
- Environment setup
- Step 1: Data exploration & staging models
- Step 2: Data modelling & dbt DAG
- Step 3: Testing & documentation
- Wrap-up

Art gallery challenge: monthly commission reports

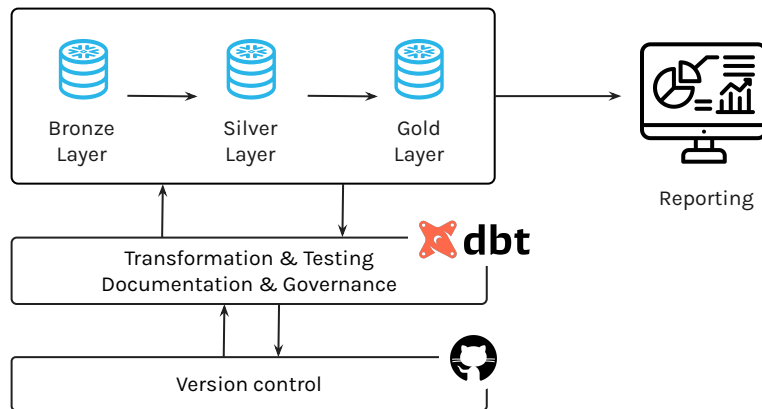
Current state

3 days of manual work every month



Our solution

Automated, reliable reporting with dbt & Snowflake



Our toolbox



- SQL-first approach
- Modularity
- Dependency management
- Version control integration
- Documentation generation
- Automated testing



- Cloud-native architecture
- Standard SQL interface
- Separation of compute and storage
- Multi-cluster architecture
- Time Travel
- Secure data sharing

Art gallery data

Core datasets

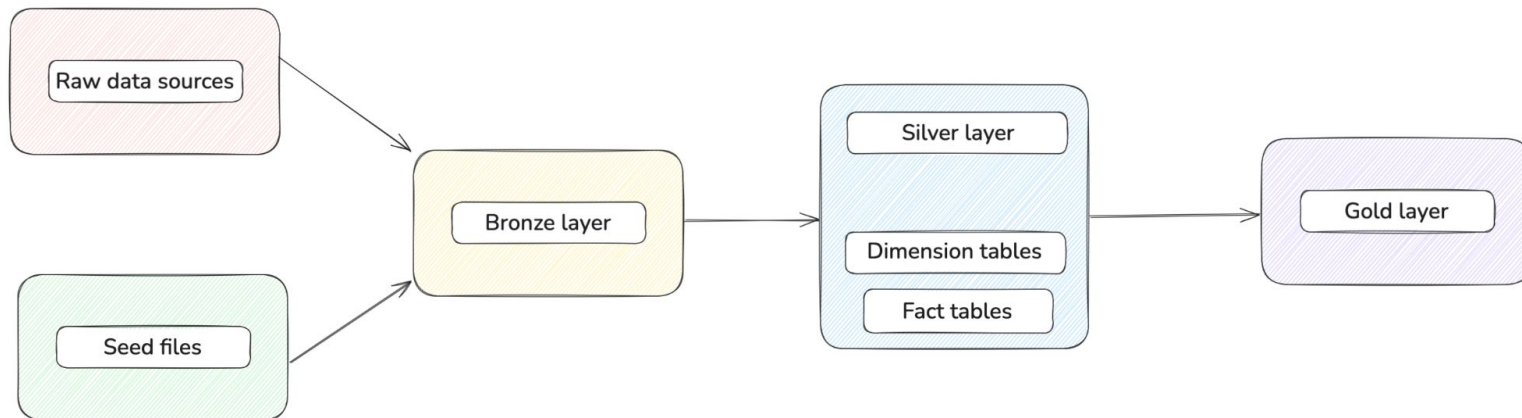
- sales_transactions
- artwork_inventory
- artist_profiles
- customer_data

Reference data

- artwork_categories
- gallery_locations
- commision_rates

Dimensional data model

- Consistency of data transformations
- Performance optimisation
- Data quality enforcement
- Automated reporting



Environment setup

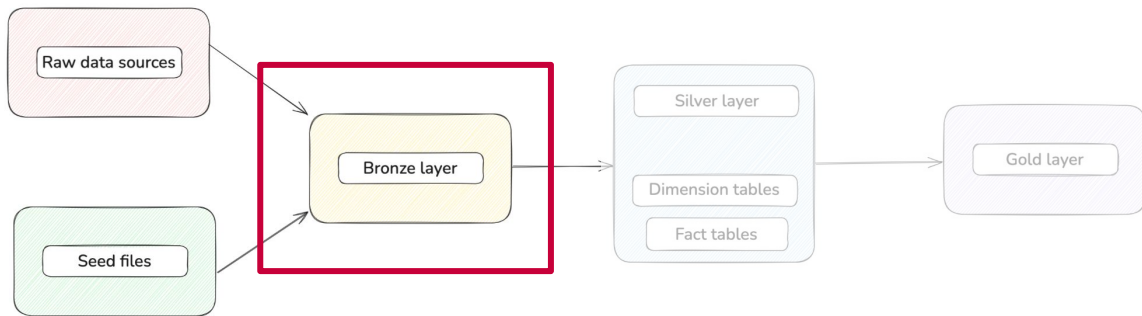
- Snowflake trial account with loaded data
- Python and virtual environment setup
- dbt Core installation and setup

Follow the [setup guide in the workshop repo](#) to get started

Exercise 1: Create **Bronze models**

Purpose:

- Rename columns to consistent conventions
- Cast data types appropriately
- Handle missing values and nulls
- Remove duplicates
- Add data quality flags
- Basic business rule validation

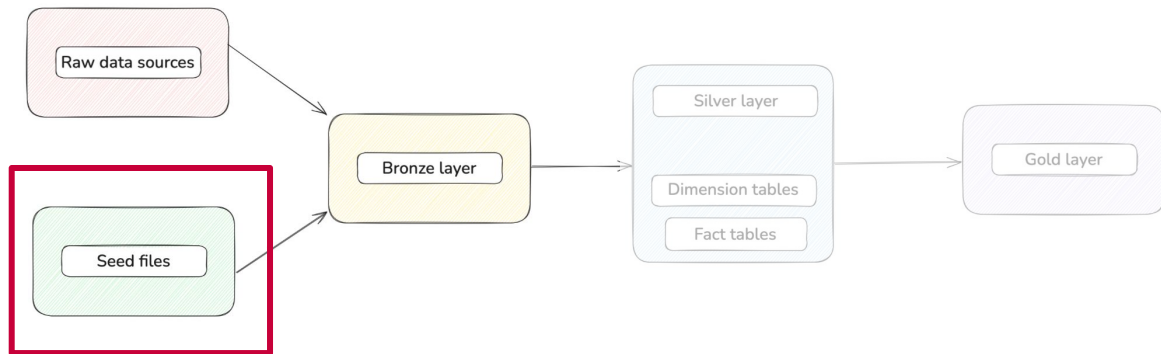


Materialisation:

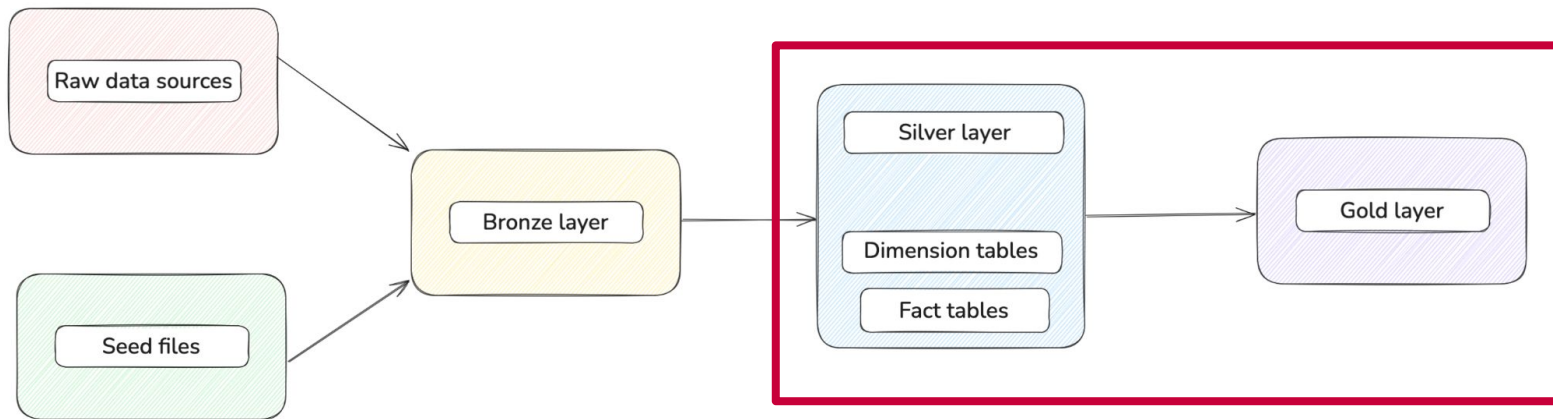
Usually views

Step 1.2: Add **seed files**

- Version-controlled reference data
- Easy to update and deploy
- Consistent across environments



Step 2: Create **Silver & Gold** models



Exercise 2: Create Silver & Gold models

Model type

Examples

Dimension tables

- Contain descriptive attributes
- Provide context for facts
- Support filtering and grouping

dim_artists

dim_artwork

dim_customers

Fact tables

- Contain measurable events (sales transactions)
- Include foreign keys to dimensions
- Store metrics and calculations

fact_sales

What is Directed Acyclic Graph?

Components

Directed: Edges have direction (one-way relationships)

Acyclic: No circular dependencies (no loops)

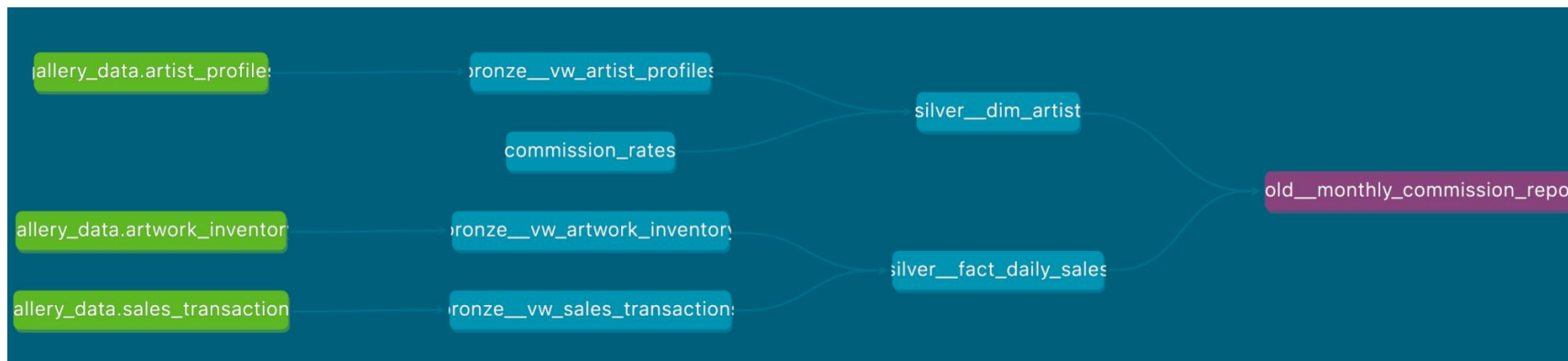
Graph: Collection of nodes connected by edges

In dbt context

Dependencies via ``ref()`` and ``source()`` functions

No circular dependencies

Models, seeds, tests, snapshots



Exercise 3: Testing & documentation

Common data problems

- Missing required values
- Duplicate records
- Invalid foreign key relationships
- Negative values where positive expected
- Future dates where past expected

dbt testing benefits

- Automated validation with standard & custom tests
- CI/CD integration
- Documentation of data contracts
- Early error detection

Useful dbt commands

dbt run	Executes the SQL in your models to build or update tables and views in your data warehouse
dbt test	Validates your data by running tests defined in your schema files against your models
dbt build	Combines running and testing operations in a single command with built-in dependency management
dbt seed	Loads CSV files from your seeds directory into your data warehouse as tables
dbt docs generate	Creates documentation for your dbt project, including model lineage diagrams and column descriptions
dbt docs serve	Starts a local web server to view the documentation generated by `dbt docs generate`

What's next?

- Real-world implementation considerations
- Integrating dbt Core with orchestration tools
- Considerations for scaling dbt projects
- Version control and CI/CD for dbt projects

Useful resources

- dbt Documentation: docs.getdbt.com
- dbt [guides & quick starts](#)
- dbt [best practices](#)
- [dbt Slack](#)
- dbt [learning portal](#)

Questions?

Get in touch:



anyalitica.dev

Thank you!



Illustration by Freepik Stories